

Java Networking

Building Connected Applications

Learning Objectives

- What We Will Learn
 - Understand fundamental networking concepts (IP, Ports, Protocols).
 - Use java.net classes for URL and URI handling.
 - Implement TCP Client-Server communication using Socket.
 - Implement UDP communication using DatagramSocket.
 - Use the modern HTTP Client (Java 11+) for web APIs.
 - Integrate Networking with Swing (Threading considerations).
 - Handle Network Exceptions (IOException, UnknownHostException).

Networking Fundamentals

- **Core Concepts**
 - **IP Address:** Unique identifier for a device (e.g., 192.168.1.1 or localhost).
 - **Port Number:** Specific endpoint on a device (0-65535).
 - **0-1023:** Well-known ports (HTTP: 80, HTTPS: 443, SSH: 22).
 - **1024+:** User-defined (Use > 5000 for custom apps).
 - **Protocol:** Rules for communication.
 - **TCP:** Reliable, ordered connection (like a phone call).
 - **UDP:** Fast, unordered packets (like sending mail).

The java.net Package

- Java Networking API
 - Core Classes:
 - InetAddress: Represents an IP address.
 - URL / URI: Represents a web address.
 - Socket / ServerSocket: TCP connections.
 - DatagramSocket: UDP connections.
 - HttpClient (Java 11+): Modern HTTP requests.
 - Exceptions:
 - IOException: General network failure.
 - UnknownHostException: IP/DNS lookup failed.
 - ConnectException: Connection refused.

Working with URLs

- URLs and URIs
 - **URL Class:** Represents a Uniform Resource Locator

```
URL url = new
URL("https://www.example.com/index.html");
System.out.println("Host: " + url.getHost());
System.out.println("Port: " + url.getPort());
System.out.println("Path: " + url.getPath());
```

- **Opening a Stream**

```
try (InputStream in = url.openStream()) {
    // Read content like a file
}
```

- **Note:** `openStream()` is blocking and simple. For complex HTTP, use `HttpClient`.

TCP Sockets (The Foundation)

- TCP Sockets: Socket & ServerSocket
 - TCP: Connection-oriented, reliable.
 - Server Side:
 1. Create ServerSocket(port).
 2. Call accept() (blocks until client connects).
 3. Get Socket from accept().
 - Client Side:
 1. Create Socket(host, port).
 2. Connects immediately.

Server-Side Code Example

- Writing a TCP Server

```
ServerSocket server = new ServerSocket(5000);  
while (true) {  
    Socket clientSocket = server.accept(); // Blocks here  
    // Handle client in a new Thread (see Slide 11)  
    InputStream in = clientSocket.getInputStream();  
    OutputStream out = clientSocket.getOutputStream();  
    // Read/Write data...  
    clientSocket.close();  
}
```

- **Key Point:** accept() blocks the thread.
- **Key Point:** Each client needs its own stream handling.

Client-Side Code Example

- Writing a TCP Client

```
String host = "localhost";
int port = 5000;

try (Socket socket = new Socket(host, port);
    PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
    BufferedReader in = new BufferedReader(new
InputStreamReader(socket.getInputStream()))) {

    out.println("Hello Server");
    String response = in.readLine();
    System.out.println(response);
}
```

- **Try-With-Resources:** Automatically closes socket and streams.
- **Streams:** Use PrintWriter and BufferedReader for text communication.

UDP Networking

- **UDP with DatagramSocket**
 - **Connectionless:** No handshake, no guaranteed delivery.
 - **Packet Based:** Data sent in DatagramPacket.
 - **Use Cases:** Streaming, Gaming, DNS.

```
DatagramSocket socket = new DatagramSocket();  
  
byte[] buf = "Hello".getBytes();  
  
DatagramPacket packet = new DatagramPacket(buf,  
buf.length, address, port);  
  
socket.send(packet);
```

Modern HTTP Client (Java 11+)

- The java.net.http Package
 - **Legacy:** HttpURLConnection (Verbose, cumbersome).
 - **Modern:** HttpClient (Async support, HTTP/2).

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/data"))
    .GET()
    .build();
HttpResponse<String> response = client.send(request,
    BodyHandlers.ofString());
System.out.println(response.body());
```

Networking & Threads

- Critical: Networking in Swing
 - **Problem:** Network operations are **blocking** (slow).
 - **Rule: NEVER** run network code on the **Event Dispatch Thread (EDT)**.
 - **Result:** UI freezes until network operation completes.
 - **Solution:**
 - Use `SwingWorker`.
 - Use `CompletableFuture` (Java 11+).
 - Use separate Thread.

```
// Correct Pattern
SwingWorker<Void, Void> worker = new SwingWorker<>() {
    protected Void doInBackground() {
        // Network code here
        return null;
    }
    protected void done() {
        // Update UI here
    }
};
worker.execute();
```

Networking & I/O

- Streams over Networks
 - Sockets provide Streams:
 - `socket.getInputStream()`
 - `socket.getOutputStream()`
 - Chaining: Wrap socket streams in higher-level streams.
 - `BufferedReader` (for lines of text).
 - `ObjectInputStream` (for serialized objects).
 - `DataInputStream` (for primitives).
 - Protocol Design: Agree on a format (e.g., JSON, CSV, or custom strings).

Security (SSL/TLS)

- **Secure Networking**
 - **HTTP vs. HTTPS:** Encryption is vital for sensitive data.
 - **SSLSocket / SSLServerSocket:** Encrypted TCP sockets.
 - **HttpsURLConnection / HttpClient:** Handles HTTPS automatically.
 - **Certificates:** Java uses a TrustStore (cacerts) to verify server identities.
 - **Warning:** Do not disable certificate validation in production.

Common Pitfalls

- **Networking Best Practices**
-  **Do:**
 - Always close sockets (use try-with-resources).
 - Handle IOException gracefully.
 - Use timeouts (socket.setSoTimeout()).
 - Run network code in background threads.
-  **Don't:**
 - Hardcode IP addresses (use DNS or config files).
 - Trust incoming data (validate input).
 - Block the EDT.
 - Ignore firewall issues (ports may be blocked).

Summary & Lab

- **Summary:**
 - TCP for reliability, UDP for speed.
 - Sockets use Streams (I/O).
 - Always background network tasks in Swing.
 - Use HttpClient for web APIs.
- **Lab Exercise:**
 - Option A (Chat): Create a simple Client-Server chat app using TCP Sockets and Swing.
 - Option B (Weather): Create a Swing app that fetches weather data from a public API (e.g., OpenWeatherMap) using HttpClient and displays it.
 - Requirement: Use SwingWorker to prevent UI freezing.